



Tech demo:

Automation of Fabric Management with Semantic Link Labs

- What is Semantic-Link-Labs?
- Capacity migration with SLL
- Workspace permission management

What is Semantic Link Labs

- SSL is a Python library that runs inside Fabric Notebooks, and provides an easy way to interact with Fabric APIs, produced by Microsoft
- It allows you to manage your Fabric ecosystem via code-first artefacts.
- Compute resources and Spark Clusters are automatically managed for you by Fabric, and taken from your Fabric SKU's CU budget

```
sempy_labs.workspace.assign_to_capacity(capacity: str | UUID, workspace: str | UUID | None = None,
**kwargs)
```

Assigns a workspace to a capacity.

This is a wrapper function for the following API: Workspaces - Assign To Capacity.

Service Principal Authentication is supported (see [here](#) for examples).

Parameters:

- `capacity (str | uuid.UUID)` – The name or ID of the capacity.
- `workspace (str | uuid.UUID, default=None)` – The name or ID of the Fabric workspace. Defaults to None which resolves to the workspace of the attached lakehouse or if no lakehouse attached, resolves to the workspace of the notebook.

Resources

- <https://github.com/microsoft/semantic-link-labs>
- <https://semantic-link-labs.readthedocs.io/en/stable/index.html>

...

I'm not going to talk at length here about Semantic Link Labs, there's some really good resources from Microsoft which I've listed down the bottom of these slides – but in short it's a Python wrapper for the Fabric APIs, and the import part of that is

- it means you can run these functions inside Fabric notebooks, the same ones you use for data engineering and machine learning
- Plus, all the authentication is default handled for you, it makes API queries as the active user, meaning you don't have to worry about token management, API authentication headers, ...

It's a super clean interface for managing your assets via code!! No folders of PowerShell scripts and confused debugging – you get to leverage all the modern data manipulation tools, and years of general python development quality of life features.

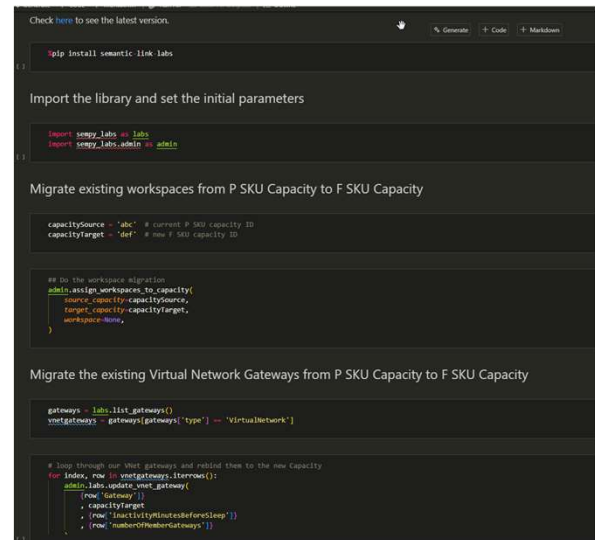
SLL for Capacity Migration

In December 2025 Microsoft put out a blog detailing how to migrate your P SKU Capacities to Fabric F SKUs.

One of the options suggested was an Automated migration tool, using Fabric Notebooks and SLL functions.

Resources

- <https://www.microsoft.com/en-us/microsoft-fabric/blog/2024/12/02/automate-your-migration-to-microsoft-fabric-capacities>
- <https://github.com/microsoft/semantic-link-labs/blob/main/notebooks/Capacity%20Migration.ipynb>



```
Check here to see the latest version.
% Generate  + Code  + Markdown

!pip install semantic-link-labs

Import the library and set the initial parameters

import smelly_labs as labs
import smelly_labs.admin as admin

Migrate existing workspaces from P SKU Capacity to F SKU Capacity

capacitySource = 'def' # current P SKU capacity ID
capacityTarget = 'def' # new F SKU capacity ID

## do the workspace migration
admin.assign_workspaces_to_capacity(
    source_capacity=capacitySource,
    target_capacity=capacityTarget,
    workspace=None,
)

Migrate the existing Virtual Network Gateways from P SKU Capacity to F SKU Capacity

gateways = labs.list_gateways()
vnetgateways = gateways[gateways['type'] == 'VirtualNetwork']

# loop through our vnet gateways and rebind them to the new Capacity
for index, row in vnetgateways.iterrows():
    admin.labs.update_vnet_gateway(
        [row['id']],
        capacityTarget,
        [row['inactivityMinutesBeforeSleep']],
        [row['numberOfWorkersPerGateway']]
    )
```

Our first job with Semantic Link Labs was to upgrade our P1 capacity to an F64, as these are no longer supported

This meant migrating all the content in Premium Workspaces to use the new capacity, ideally without disruption, data loss or any need to reupload files

Sure, you can do this through the Admin GUID or in Workspace settings, but if there's APIs for it, and we can submit a change plan ahead of time, it makes the job way easier, more reliable and removes human error

Luckily for us, Microsoft had already done most of the heavy lifting

SLL for Capacity Migration

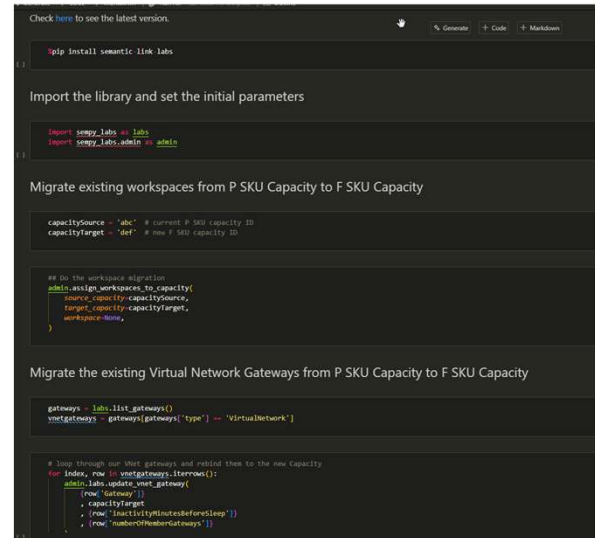
Department of Justice used this as a template, and extended it slightly to migrate both Workspaces, and existing V-Net Gateways.

In total we migrated 62 Workspaces, 5 V-Net Gateways, and 7,101 associated items.

Took under 5 minutes

Resources

- <https://www.microsoft.com/en-us/microsoft-fabric/blog/2024/12/02/automate-your-migration-to-microsoft-fabric-capacities>
- <https://github.com/microsoft/semantic-link-labs/blob/main/notebooks/Capacity%20Migration.ipynb>



```
Check here to see the latest version.
% Generate  + Code  + Markdown

!pip install semantic-link-labs

Import the library and set the initial parameters

import smgy_labs as labs
import smgy_labs.admin as admin

Migrate existing workspaces from P SKU Capacity to F SKU Capacity

capacitySource = 'def' # current P SKU capacity ID
capacityTarget = 'def' # new F SKU capacity ID

## do the workspace migration
admin.assign_workspaces_to_capacity(
    source_capacity=capacitySource,
    target_capacity=capacityTarget,
    workspace=None,
)

Migrate the existing Virtual Network Gateways from P SKU Capacity to F SKU Capacity

gateways = labs.list_gateways()
vnetgateways = gateways[gateways['type'] == 'VirtualNetwork']

# loop through our vnet gateways and rebind them to the new Capacity
for index, row in vnetgateways.iterrows():
    admin.labs.update_vnet_gateway(
        [row['id']],
        capacityTarget,
        [row['inactivityMinutesBeforeSleep']],
        [row['numberOfReservations']]
    )
```



Workspace permission management

Workspace permission management

Administration & Security challenge: Maintaining desired access

- Ideally Workspaces permissions are controlled through EntraID Security groups, and there are no user permissions.
- Only approved users have access.
- However, “Member” role allows users to add others, bypassing the EntraID group, and any associated approvals / logging

This table shows a small set of the capabilities each role has. For a full and more detailed list, see [Microsoft Fabric workspace roles](#).

Capability	Admin	Member	Contributor	Viewer
Delete the workspace	✔	✗	✗	✗
Add admins	✔	✗	✗	✗
Add members	✔	✔	✗	✗
Write data	✔	✔	✔	✗
Create items	✔	✔	✔	✗
Read data	✔	✔	✔	✔

Semantic Link Labs allows us to programmatically assign, and remove Users, Groups and machine agents from Items, Workspaces, Capacities, ...

Resources

- <https://learn.microsoft.com/en-us/fabric/security/permission-model>

Okay so now that we’ve used Semantic Link Labs, I thought how can we bring this “platform / infrastructure as code” type approach to other parts of my Fabric Admin job

One of the main areas of overhead for me is the setting up, and management, of Workspaces for our many-many business areas, across different environments, different delivery patterns, different levels of maturity...

At a platform level, we’re aiming for group-based access as a default, no sea of hundreds of customized user permissions – instead it should be all Role Based Access, and ideally managed as code, and subject to approvals or PIM.

This means when staff move or join, we add them to a few groups, and their access updates across all our systems, Fabric, PBI, Teams, Databricks, ...

However, in Fabric, often developer permissions allow you to add other people – and if users do this instead of adding staff to groups, this bypasses logging, it skips approval steps, it means people don’t get access to other resources they’ll need... and we in central IT also have a duty to ensure there’s approved access only!

Workspace permission management



This approach has been blatantly ripped from a colleague of mine who implemented a similar approach for Power BI Connector access management.

Our goals were:

- Access is always aligned to an approved target state
- Access is pro-actively audited and remediated
- Access management doesn't have to be done manually
- Access can be re-provisioned automatically if it's lost
- To follow an Infrastructure As Code style approach
- Stay Azure native
- Minimal cost
- *Maintain minimal technical overhead*

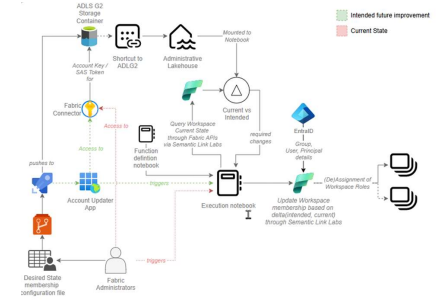
Resources

- <https://learn.microsoft.com/en-us/fabric/security/permission-model>
- <https://www.linkedin.com/in/stephen-rob-price/>

Workspace permission management

Essentially the core logic is:

1. Load what permissions we want
2. Query what they are now
3. Evaluate the difference using Python dataframe operations
4. Act on the variance
 - i. Add Users, Groups, Principals
 - ii. Remove Users, Groups, Principals
5. Let us know what's been done



Resources

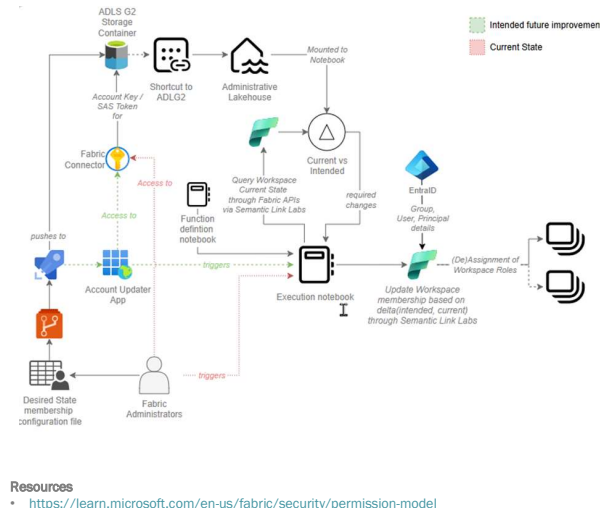
- <https://learn.microsoft.com/en-us/fabric/security/permission-model>

Workspace permission management

Started off quite simple:

This may work for your organisation, if you accept:

- No private networks,
- Activated / persistent admin roles
- User owned assets



Components:

- Target state file (.csv)
- Azure Storage Container
- Fabric Capacity
- Fabric Connector
- Fabric Workspace
- Fabric Lakehouse
- Lakehouse Shortcut
- Fabric notebook (Utilities)
- Fabric Notebook (execution)

The initial build of this solution was to

- Load the configuration directly to a container
- Set up a Lakehouse to query that container
- Call Semantic Link Labs functions from a notebook to get the current and target state, then make the changes

This all operated with my super user account, and only ever had a “production mode”

10

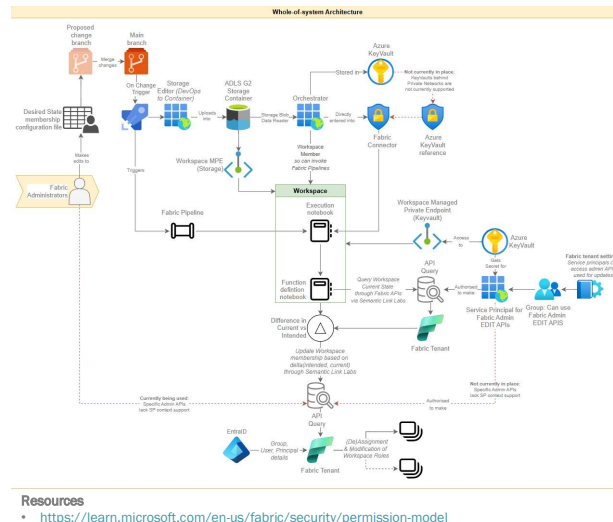
Workspace permission management

It got much more complex:

Due to governance & networking requirements, we had to add a lot

Now it's

- Behind private networks
- Run by Service Principals not users – *I can go on holiday*



Components:

- Target state file (.csv)
- Azure Storage Container
- Azure Key-vault
- Fabric Capacity
- Fabric Connector
- Fabric Workspace
- Fabric notebook (Utilities)
- Fabric Notebook (execution)
- Fabric Pipeline (orchestration)
- Fabric Administrator rights
- Service principals
- Private networks
- Managed Private Endpoints

Essentially what's happening here is

1. There's security around the data container
2. We use a Connector to hold secrets which allow reading from the container – because Keyvault references don't play nicely with private networks, we have to enter that key every 6 months both in Keyvault and in Fabric – references would cut out the Fabric step
3. That container is connected live to a Lakehouse using a Fabric Shortcut
4. There's a few notebooks which define configurations, environment variables for test modes, which load the data from the Lakehouse, and which define all our critical functions
5. Inside those, there's a query out to Keyvault, to get the admin principal details, and that also has to go through network security
6. From there we have a notebook which contains the logic, and a pipeline which executes the notebook, allowing it to be run from a principal, not a user account

Workspace permission management

It's actually very slim*

```
1 $run fabric-workspace-membership-validation-util
[1] ✓ - Command executed in 1 min 1 sec 367 ms by Jack MacCormick (51) on 04/03/2026, 10:02:01
PySpark (Python)

***
Output is hidden

1 # this will loop through all workspaces in the managed workspace list (see lakehouse), and enforce the desired access configuration - eliminating any permission drift
2 for workspace_id in workspaces:
3     enforce_workspace_membership(workspace_id)
- Command executed in 9 sec 556 ms by Jack MacCormick (51) on 04/03/2026, 10:02:11
PySpark (Python)

1 notebookutils.notebook.exit("Success")
[1] ✓ - Command executed in 1 sec 362 ms by Jack MacCormick (51) on 04/03/2026, 10:02:12
PySpark (Python)

ExitValue: Success
```

* I'm cheating quite a lot here by putting the function definitions & environment prep code into another file, and loading those in the first cell

As you can see here, the actual code is really short!!

Okay that's a bit of a lie,

I've moved all the complicated logic into upstream notebooks as functions, so that we can call *"manage this"* against a workspace, and this script specifically just loops through all the ones we're looking after.

Moving the prep work upstream means we can be really efficient at targeting specific resources, and we don't need to copy paste big chunks of code from the last job- if something needs to be updated, its done in one place, and everything downstream just picks that up

Workspace permission management

The actual interesting code looks a lot like this, and is essentially

1. Check if there's any difference
2. Split groups from other entities
3. Loop through what needs to be done, using SLL functions to do the work
4. Flag what steps we did

There's some error handling and credential management that's done elsewhere

```
1
2 # do we need to remove anyone?
3 if pd.DataFrame(to_remove).size == 0:
4     print("no removals needed")
5 else:
6     # logic to go here
7     users_sp_to_rm = to_remove[to_remove['Principal Type'] != "Group"]
8     groups_to_rm = to_remove[to_remove['Principal Type'] == "Group"]
9     print("start removing users and principals")
10    for index, row in users_sp_to_rm.iterrows():
11        labs.admin.delete_user_from_workspace(
12            row['Identifier'],
13            row['workspace'])
14    )
15    print("removed all users and principals")
16    print("removing groups")
17    for index, row in groups_to_rm.iterrows():
18        labs.admin.delete_user_from_workspace(
19            row['Identifier'],
20            row['workspace'])
21        , True
22        )
23    print("removed all groups")
24    print("end remove logic")
```

As you can see here, the actual code is really short!!

Okay that's a bit of a lie,

I've moved all the complicated logic into upstream notebooks as functions, so that we can call "*manage this*" against a workspace, and this script specifically just loops through all the ones we're looking after.

Moving the prep work upstream means we can be really efficient at targeting specific resources, and we don't need to copy paste big chunks of code from the last job- if something needs to be updated, its done in one place, and everything downstream just picks that up

I'm looking at making this available, but I need to discuss that with Justice a bit first



Live Demo

Cheers!

Jack MacCormick

<https://www.linkedin.com/in/jack-maccormick-datasci/>

<https://github.com/Argel-Tal?tab=repositories>

